Quiz 3 · 14 questions

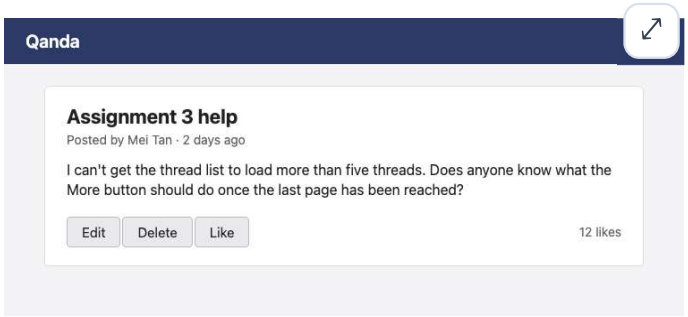
Delete beside Edit

3 marks

On the Qanda discussion site, the creator of a thread sees three buttons under it, as in the screenshot thread.png. Edit lets the creator change the thread. Delete removes the thread straight away and sends the user back to the dashboard. Like adds a like. Mei is the creator of this thread and wants to edit it.

Supporting files

thread.png



(a) What about the design of these three buttons makes it easy for Mei to press Delete by mistake? 1 mark

All three buttons look the same (same colour, size and style) and sit right next to each other with almost no gap, so nothing sets Delete apart from Edit and Like, and a slightly off click lands on the wrong one.

(b) Describe one change that makes an accidental delete less likely, and one different change that makes it less costly when it still happens. 2 marks

Less likely: ask "Delete this thread?" and wait for a confirmation before deleting; style Delete differently from the other buttons (for example red or outlined); or move or space Delete away from Edit and Like. Less costly: let the user undo the delete straight afterwards, or keep deleted threads somewhere they can be restored from.

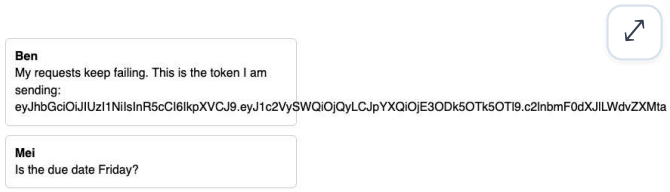
A token that runs out of its comment

3 marks

On the Qanda thread page each comment is shown in a box. Ben pastes a long login token into his comment. The markup and styles are in index.html and style.css, and comments.png is a screenshot of the page.

Supporting files

- comments.png
- index.html
- style.css



(a) Ben's sentence wraps onto new lines inside the box, but the token runs straight out of it. Why does the token not wrap too?

1 mark

The browser only breaks lines at places where it is allowed to, such as spaces (and hyphens). The token is one very long "word" with no spaces in it, so there is nowhere to break it. It is wider than the 360px box, and because the box has a fixed width it cannot grow, so the token overflows past its edge.

(b) A teammate suggests adding `overflow: hidden` to `.comment`. Describe what Ben's comment would look like then, and give a better CSS fix with the reason it is better.

2 marks

With `overflow: hidden`, the overflow is removed, but the end of the token would be cut off at the right edge. Other users will not see or copy the whole token.

A better fix is `overflow-wrap: anywhere` (or `overflow-wrap: break-word`, or `word-break: break-all`) on the comment text. It allows the browser to break inside a word when the word cannot fit on a line, so the token wraps onto several lines inside the box [illegible]

Buttons that jump

3 marks

A Qanda post contains a 600 by 300 pixel sketch, with Reply and Like buttons underneath. The markup and styles are in index.html and style.css. Mei is on slow Wi-Fi. loading.png shows the page just after it appears, and loaded.png shows it about two seconds later, after the sketch has downloaded. Mei went to click Reply as soon as the page appeared, but her click landed on the sketch.

Supporting files

- index.html
- loaded.png
- loading.png
- sketch.png
- style.css

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>Qanda</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h2>How should the dashboard look?</h2>
  <p>Here is my sketch:</p>
  <img src="sketch.png" alt="Sketch of the
  <div>
    <button>Reply</button>
    <button>Like</button>
  </div>
</body>
</html>
```

(a) Why do the Reply and Like buttons move down when the sketch finishes downloading?

1 mark

The `<img>` tag does not define picture size, so the browser doesn't know its size until the file is downloaded. Until then it gives the image no space, and the buttons just sit under the text. When the 600 by 300 picture arrives, the browser makes room for it and pushes everything below it down by about 300px.

(b) Describe a change to the page that stops the buttons from moving, and explain why it works.

2 marks

Add the picture's size to the tag: `<img src="sketch.png" width="600" height="300" alt="...">`. The browser then knows how big the image will be before the file arrives, so it reserves a 600 by 300 box straight away. The buttons start in their final position and nothing moves when the picture appears.

An empty dashboard

3 marks

dashboard.png shows the Qanda dashboard for a user who has just registered and has no threads yet. The only way to create a thread is the "+" in the header. In a usability test, 8 new users were asked to start a discussion. 5 said the page "was still loading" or "looked broken". The other 3 found the "+", but only after more than a minute.

Supporting files

dashboard.png



Dashboard

(a) Every tester's dashboard was working correctly, but they did not realise it. Why not, and what might they have expected to see that would show the page was working?

1 mark

The blank area below "Dashboard" looks the same whether the threads are loading, failed to load, or there are no threads. Nothing on the page tells these apart, so testers assumed the list had not arrived yet or had failed. They would have expected the page to say what state it is in: a message such as "You have no threads yet" once the list has loaded empty (or a "Loading..." indicator while it is still loading).

(b) Describe how the "+" for creating a thread could be shown instead so that new users find it easily, and give an example HTML element, with CSS, for your idea. Inline CSS is accepted.

2 marks

Replace the small icon-only "+" with a clearly labelled button, such as "Create your first thread", placed in the empty area below "Dashboard" where new users are already looking. Style it as the main action on the page: larger, with padding and a filled background in the site's colour, so it stands out. The text label says what it does, so users do not have to guess what "+" means. For example:

```
<button style="padding: 12px 24px; font-size: 18px; background: #2b3a67; color: white; border: none; border-radius: 6px;">
  + Create your first thread
```

Helpers for the register page

3 marks

The Qanda team keeps two helper functions for the register page in helpers.js. parseReply turns the server's JSON reply into a list of thread titles. showError puts a message into the page's error popup. In the page, helpers.js is loaded with a script tag in the head, above the div with the id error-container. runtime-order.svg shows the Node test calls and the browser's document order.

Supporting files

helpers.js

runtime-order.svg

```
function parseReply(text) {
  const reply = JSON.parse(text);
  return reply.threads.map((thread) => thread.t
}

function showError(message) {
  document.getElementById('error-container').te
}
```

(a) The team wants to check these functions automatically with Node, without opening a browser, by calling each one with test input. One function can be checked this way and the other cannot. Which one cannot, and why?

1 mark

showError cannot. parseReply only uses JSON.parse and .map, which are part of the JavaScript language and so work in any runtime, including Node. showError uses `document`, which is added by the browser and does not exist in Node, so calling it stops with `document is not defined`.

(b) On the register page, a teammate adds the line showError('Welcome back'); at the bottom of helpers.js so that it runs as soon as the file loads. What happens on the page, and why? 1 mark

The call fails with an error (`Cannot set properties of null`) and no message appears. The script is in the head, so it runs before the browser has built the page body. The error-container div does not exist yet, so getElementById returns null and there is nothing to put the text into.

(c) Instead, the teammate leaves the script tag in the head and calls showError('Passwords do not match.') from the register form's submit handler. A user submits mismatched passwords. Does the message appear, and why? 1 mark

Yes, the message appears. The handler only runs when the user submits the form, long after the page has loaded, so by then the error-container div exists. Where the script tag sits matters only for code that runs immediately when the file loads.

Super like

6 marks

Each Qanda thread shows its like count with two buttons. "Super like" is meant to add 2 likes. page.png shows the buttons for a thread with 12 likes. LikeButtons.jsx contains the code.

Supporting files

LikeButtons.jsx

page.png

```jsx
const LikeButtons = ({ initialLikes }) =
  const [likes, setLikes] = useState(initia

  const like = () => {
    setLikes(likes + 1);
  };

  const superLike = () => {
    like();
    like();
  };

  return (
    <div>
      <p>{likes} likes</p>
      <button onClick={like}>Like</button>
      <button onClick={superLike}>Super lik
```

```
18        </div>
19      );
20    };
```

(a) The thread has 12 likes. The user clicks "Super like" once. What number is displayed, and why? 2 marks

13. During this render `likes` is the constant 12. Both calls to `like` run `setLikes(likes + 1)` with that same 12, so both ask React to set the value to 13. The second call does not see the first call's result.

(b) A teammate adds `console.log(likes)` as the last line of `superLike`. What does it print on that click, and why? 2 marks

12. Calling `setLikes` does not change the `likes` variable inside the function that is already running; it asks React to render again with the new value. Only the next render's `likes` is 13.

(c) Describe a change so that both buttons work as described, and explain why it works. 2 marks

Change `like` to use the updater form: `setLikes(prev => prev + 1)`. React runs queued updaters one after another, passing each one the result of the previous one, so two calls give 12 → 13 → 14. Also accepted: `superLike` calls `setLikes(likes + 2)` directly, with the explanation that a single update from the current value is correct.

Account redirect effect

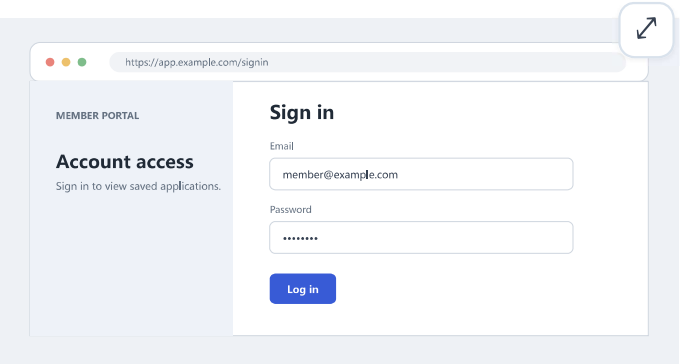
6 marks

`SignInPage` stores the authenticated user in state. The page is displayed when the user is on the URL `/signin`. The page should move to `/account` after a successful sign-in and should not show the account page to a logged-out user. SignInPage.jsx contains the component, and signin-start.svg shows the page before the form is submitted.

Supporting files

signin-start.svg

SignInPage.jsx



(a) After the sign-in request succeeds and `setUser` runs, what does the user see and what URL are they still on? Explain why this happens. 3 marks

The user remains on the sign-in page at `/signin`; the account page is not shown. The effect ran after the initial render, when `user` was `null`, and its empty dependency array means it does not run again after `setUser(result.user)` updates the state. The navigation is therefore never performed.

(b) Describe the change that makes the redirect happen whenever the authenticated user becomes available, and describe what should happen when the user logs out. 3 marks

The effect should depend on `user`, so it runs after the user state changes. When `user` is non-null it should navigate to `/account`. Logout should clear `user`; protected routing should then redirect the unauthenticated user to `/signin` rather than leaving the account page accessible.

Interval with stale state

6 marks

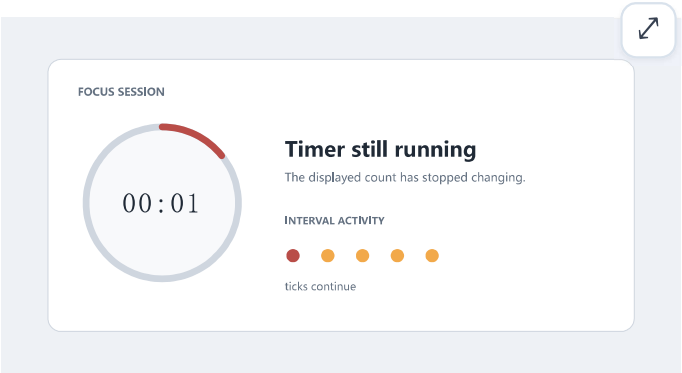
Timer.jsx displays a focus timer. It reaches 1 second and then stops changing even though the browser keeps running the interval. The app is running without React

Strict Mode.

Supporting files

timer-result.svg

Timer.jsx



(a) Timer's interval keeps running, but the displayed count stops at 1. Which value of `count` does the interval function keep, and what value does it ask React to set on every tick? 3 marks

React creates the interval function during the first render, when `count` is 0. That function keeps using 0, so every tick calls `setCount(0 + 1)`. The first tick displays 1, and later ticks keep setting the same value.

(b) Timer should create one interval while it is on screen and stop it when the user leaves. How can each tick update from the latest count without recreating the interval, and what must cleanup do? 3 marks

Use the setter form that receives the latest previous value, such as `setCount(previous => previous + 1)`. React supplies the current count to that function on each tick. The effect should keep the interval identifier and return cleanup that passes it to `clearInterval` when Timer leaves the screen.

An async effect callback

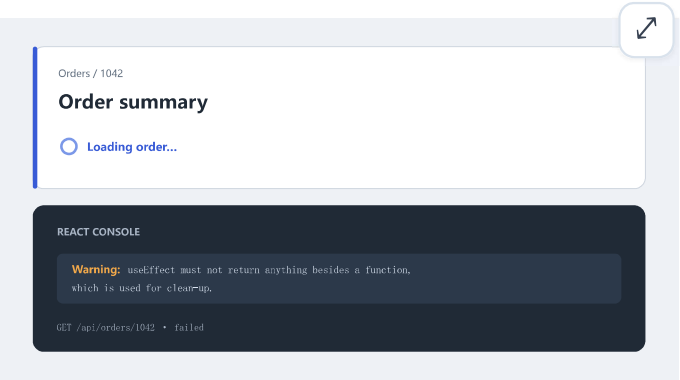
6 marks

OrderPanel.jsx loads an order summary. React prints a warning when the effect runs, and a failed request leaves the loading message on screen.

Supporting files

effect-warning.svg

OrderPanel.jsx



(a) OrderPanel has two symptoms: React warns about the function passed to `useEffect`, and a failed request leaves “Loading order...” on screen. What separate cause produces each symptom? 3 marks

An `async` function returns a Promise, so the function passed to `useEffect` returns a Promise. React expects that function to return nothing or a cleanup function. Separately, a failed fetch or JSON parse jumps past `setLoading(false)`. The code has no failure handling, so loading remains true.

(b) How should the effect be reorganised so it can keep the `await` style, turn loading off after success or failure, and stop an older order request from updating the panel? 3 marks

Pass a regular function to `useEffect`, then define and call an async function inside it. That inner function can use `try` for the request, `catch` to store an error, and `finally` to turn loading off. The effect's cleanup should abort the older request or mark it as unable to update state. The async function should change order, error, or loading only for the current `orderId`.

Hotel route parameters

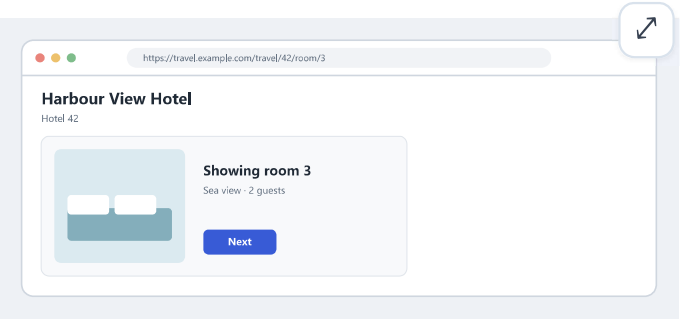
6 marks

The travel browser uses React Router so a hotel and its selected room can be opened directly or restored after a refresh, by using the URL. room-page.svg shows the page at `/travel/42/room/3`. TravelBrowser.jsx contains the route and the component.

Supporting files

room-page.svg

TravelBrowser.jsx



(a) A user refreshes `/travel/42/room/3`. What room does the component render, and what information is missing from its current route setup? 2 marks

It renders room 1, because the component always initialises `roomNumber` to 1. The route declares a `:roomNumber` parameter, but the component does not read that parameter and use it as its initial or controlled room number.

(b) The user clicks "Next" on room 3. Why is changing only React state not enough for the desired behaviour of the app? 2 marks

The visible room may change, but the URL still says room 3. A refresh or a copied link will therefore lose the user's position, because the route is the source of the room number for a directly addressable page.

(c) Describe how the component should use the route when it loads and when "Next" is clicked. Do not write code. 2 marks

Read both `hotelId` and `roomNumber` from the matched route and render the room identified by those values. On a button click, navigate to the same hotel with the next room number in the URL; the route change then causes the displayed room to update.

Navbar links

6 marks

A Qanda-style app keeps the logged-in user's token in React state. The login page calls `setToken` after a successful login and navigates to `/dashboard`. qanda-pages.svg shows the dashboard and profile pages with the navbar. App.jsx contains the code.

Supporting files

App.jsx

qanda-pages.svg

```jsx
const App = () => {
  const [token, setToken] = useState(null);

  return (
    <BrowserRouter>
      <nav>
        <a href="/dashboard">Dashboard</a>
        <a href="/profile">Profile</a>
      </nav>
      <Routes>
        <Route path="/login" element={<Logi
        <Route
          path="/dashboard"
          element={token ? <Dashboard /> :
        />
        <Route
          path="/profile"
          element={token ? <Profile /> : <N
        />
      </Routes>
    </BrowserRouter>
```

(a) Mei logs in and lands on `/dashboard`. She clicks "Profile" in the navbar and is sent to `/login` instead of her profile. Explain what the browser does when she clicks the link, and why that logs her out.

2 marks

An `<a href>` makes the browser do a full page load: it requests `/profile` from the server and starts the whole app again from scratch. All React state is thrown away, so `token` is back to `null`, and the `/profile` route sends her to `/login`.

(b) Describe a change to the navbar that keeps Mei logged in when she clicks between pages, and explain what is different about how it navigates. 2 marks

Use React Router's `<Link to="/profile">` (and `<Link to="/dashboard">`) instead of `<a href>`. `Link` stops the browser's normal page load, updates the URL, and lets the router render the matching route inside the same running app. `App` is never restarted, so `token` survives.

(c) Even with that fix, Mei is still sent to `/login` whenever she presses the browser's refresh button. Why is a refresh different from clicking a `Link`, and what else needs to change? 2 marks

Refreshing always does a full page load, which `Link` avoids, so the React state, including `token`, is lost again no matter how she navigated. The token has to be stored somewhere that survives a reload, such as `localStorage`, and read back when the app starts (for example as the initial value of the state).

Profile pages

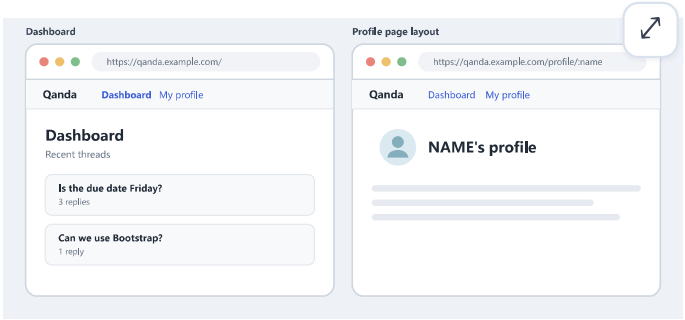
6 marks

The Qanda app has profile pages. `/profile/:name` shows that user's profile, and the "My profile" link in the navbar goes to `/profile`. profile-pages.svg shows the dashboard navbar and the profile page layout. Profile.jsx contains the code.

Supporting files

profile-pages.svg

Profile.jsx



(a) The user opens `/profile/emily`. What does the page show, and how does `Profile` know which name to show? 2 marks

"EMILY's profile". `/profile/:name` matches it: the `:name` part is a wildcard that matches any value in that position. `useParams` returns an object of the wildcard values from the URL, here `{ name: 'emily' }`, so `name` is "emily".

(b) The user clicks "My profile", which goes to `/profile`. What happens, and why? 2 marks

The page goes blank and the console shows an error like "Cannot read properties of undefined (reading 'toUpperCase')". `/profile` matches the route that renders the same `Profile` component, but that route has no `:name` wildcard, so `useParams()` returns an empty object and `name` is `undefined`. Calling `toUpperCase` on `undefined` throws while rendering, and React removes the page.

(c) A teammate's fix is to delete the `/profile` route. What does `/profile` show now, and why? Then describe a better fix. 2 marks

Nothing from the routes (only the navbar is displayed). React Router matches a route's whole path, and `/profile` does not match `/profile/:name`, which needs a value after `/profile/`. No route matches, so nothing is rendered for the page. A better fix is to keep both routes and have `Profile` check whether `name` exists: if it does not, render something else, such as "Please enter a name" with a search box, or the logged-in user's own profile.

Vote totals in two components

6 marks

A live poll shows two totals on the same page. vote-result.svg shows the values after a vote. PollPage.jsx contains the relevant parent and button.

Supporting files

PollPage.jsx

vote-result.svg

```jsx
const PollPage = ({ poll }) => (
  <>
    <ResultSummary votes={poll.votes} />
    <VoteButton startingVotes={poll.votes}
  </>
);

const VoteButton = ({ startingVotes, pollI
  const [votes, setVotes] = useState(starti

```

```
11    const vote = async () => {
12      await sendVote(pollId);
13      setVotes(votes + 1);
14    };
15
16    return <button onClick={vote}>Vote ({vote
17  };
```

(a) After a vote succeeds in PollPage.jsx, the button changes to "Vote (13)" but the summary above it stays at 12. Which value does each component read, and why do the totals differ? 2 marks

VoteButton increments its own local `votes` state from 12 to 13. ResultSummary still receives `poll.votes` from the unchanged parent prop, so it continues to display 12.

(b) Where should the total state and vote action live so the button and summary cannot drift apart? 2 marks

The nearest common parent, PollPage, should own the current poll or total and pass that value to both children. VoteButton should receive an action callback instead of creating a second copy of the total.

(c) Suppose the server rejects the next vote. What should remain on screen, and what feedback would allow another attempt without inventing an unconfirmed total? 2 marks

The parent should keep or restore the last confirmed total rather than permanently counting the rejected vote. It should show a visible recoverable error and allow another attempt; a temporary optimistic value is acceptable only if it is rolled back on failure.

Two cart item counts

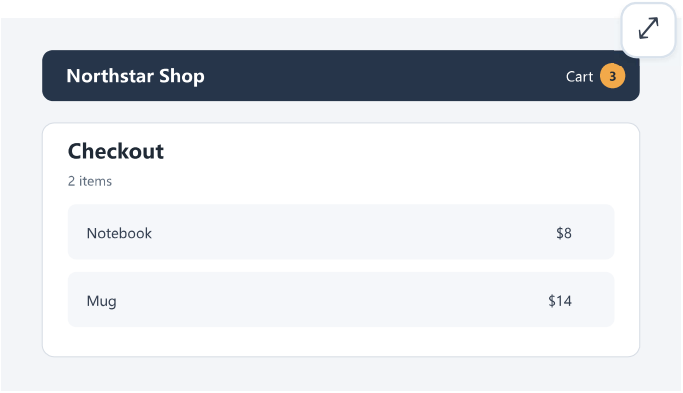
6 marks

The cart header and checkout show different item counts. cart-result.svg shows the page after an item is removed. CartApp.jsx shows where each branch creates its state.

Supporting files

cart-result.svg

CartApp.jsx



(a) A checkout action removes an item, so its count becomes 2, but the header badge still says 3. Why did the matching initial values in CartApp.jsx not stay linked? 2 marks

HeaderCart and Checkout each created a separate state value from `initialCart`. Updating Checkout's copy does not update HeaderCart's independent copy, even though both started from the same data.

(b) Where should the cart state live, and how can the header and checkout receive it when they sit in different branches of the component tree? 2 marks

The cart should be owned above both branches, such as in their common application parent. It can be passed through props or supplied through context at that level when prop passing is impractical.

(c) After the quantity endpoint confirms a change, how should the owner update the cart so both displays agree without mutating old state? 2 marks

It should replace the changed item inside a new cart array, or replace the whole cart with the server's confirmed result. Both the badge and checkout must derive from that one new state value.